BÁO CÁO BÀI LAB3

Đề 1: Thiết kế mạch giám sát và cảnh báo người chạy

Sinh viên: Nguyễn Thành Trung

MSSV: 231432378

Đề 1.

Thiết kế Mạch Giám sát và cảnh báo người chạy (runner). Mạch này sử dụng ESP32 đọc dữ liệu nhiệt độ và áp lực khí quyển => độ cao (so với mực nước biển) từ cảm biến BMP180, hiển thị dữ liệu qua giao tiếp serial. Đồng thời ESP32 gửi dữ liệu lên MQTT Broker qua Wifi và hiển thị dữ liệu trên Web/App.

Yêu cầu thực hiện:

• Vẽ mạch kết nối trong Wokwi.

• Thiết kế giao diện trên Web/App với các dữ liệu đọc được từ Broker hiển thị trên thẻ html "class" và nút "button" để điều khiển cơ cấu chấp hành:

+ Các thẻ html hiển thị dữ liệu nhiệt độ, áp lực khí quyển và độ cao (so với mực nước biển) dạng số.

+ 2 biểu đồ cập nhật dữ liệu nhiệt độ và độ cao, trực tiếp theo thời gian thực (tùy chọn).

+ 2 nút button để điều khiển trạng thái Led và Loa cảnh báo (chế độ manual).

• Xây dựng hệ RTOS cho hoạt động của ESP32, với các yêu cầu:

+ "***Task truyền data***": kết nối Wifi chạy Core 0, nếu thành công, đọc dữ liệu từ hàng đợi "***data Queue***", và gửi lên MQTT broker.

+ "***Task đọc data***", chạy Core 1: đọc dữ liệu cảm biến BMP180, hiển thị dữ liệu qua giao tiếp serial, gửi dữ liệu đầy đủ vào hàng đợi "***data Queue***" và dữ liệu cảnh báo gồm nhiệt độ + độ cao vào hàng đợi "***data Alert***".

Lời mở đầu

Bài Lab xây dựng một mạch giám sát môi trường cho tình huống người chạy. ESP32 đọc nhiệt độ và áp suất từ BMP180, ước tính độ cao so với mực nước biển và bật cảnh báo led và còi khi thỏa mãn điều kiện, đưa số liệu ra Serial, phát lên MQTT và để trang web hiển thị. Trang web hiển thị dữ liệu nhiệt độ , độ cao so với mực nước biển và áp suất khí quyển và điều lệnh bật, tắt (Chế độ manual) hoặc trả LED và còi về chế độ tự động.

Mục lục

Lời mở đầu 1
Mục lục 2
Danh mục hình ảnh 3
Danh mục bảng biểu 4
Danh mục từ viết tắt 4
Chương 1. Tổng quan về RTOS và FreeRTOS 6
1.1. Hệ điều hành thời gian thực 6
1.2. Ưu điểm của RTOS 7
1.3. FreeRTOS và ưu/nhược điểm 8
Chương 2. Giao tiếp và đồng bộ giữa các task 9
2.1. Vì sao các task phải trao đổi dữ liệu 9
2.2. Cờ hiệu (Semaphore) 9
2.3. Hàng đợi (Queue) 13
Chương 3. Thiết kế và xây dựng hệ thống 16
3.1. Tổng quan 16
3.2. Phần cứng và sơ đồ Wokwi 16
3.3. BMP180 và cách tính độ cao 21
3.4. Các Task được sử dụng 23
3.5. Lưu đồ thuật toán 24
3.6. Thiết lập điều kiện cảnh báo và chế độ MANUAL/AUTO 28
Chương 4. MQTT và giao diện web 29
4.1. MQTT trong hệ thống 29
4.2. Wi-Fi, broker và topic 29
4.4. Giao diện web (index.html, style.css và app.js) 30
4.5. Code chương trình (Lab3_de1.ino và alert_logic.h) 31
Chương 5. Kết quả mô phỏng 31

Danh mục hình ảnh

• Hình 1. Kiến trúc hệ điều hành RTOS

• Hình 2. tổng quát cơ chế Semaphore trong RTOS

• Hình 3. tổng quát cơ chế Queue trong RTOS

• Hình 4. Kiến trúc các task, queue, broker và web.

• Hình 5. Kiến trúc truyền thông MQTT của hệ thống

• Hình 6. Dashboard của RUNNER MONITORING SYSTEM.

• Hình 7. Nhiệt độ từ 10 °C đến 32 °C và độ cao không đổi (Led (off), còi (off)).

• Hình 8. Nhiệt độ Lớn hơn 32 °C và nhỏ hơn hoặc bằng 35 °C và độ cao không đổi (Led (on), còi (off)).

• Hình 9. Nhiệt độ lớn hơn 35 °C và độ cao không đổi (Led (on), còi (on)).

• Hình 10. Nhiệt độ nhỏ hơn 10 °C và độ cao không đổi (Led (on), còi (on)).

• Hình 11. Nhiệt độ từ 10 °C đến 32 °C và Độ cao đổi từ 5 m đến dưới 10 m trong khoảng 5 giây (Led (on), còi (off)).

• Hình 12. Nhiệt độ từ 10 °C đến 32 °C và Độ cao đổi từ 10 m trở lên trong khoảng 5 giây (Led (on), còi (on)).

• Hình 13. Nhiệt độ từ 10 °C đến 32 °C và độ cao không đổi dùng chế độ manual bật (Led (on), còi (on)).

• Hình 14. Nhiệt độ trên 32 °C và độ cao không đổi dùng chế độ manual tắt (Led (off), còi (off)).

Danh mục bảng biểu

• Bảng 1 đến Bảng 12 lần lượt trình bày ưu nhược điểm FreeRTOS, semaphore và queue, chân GPIO, phân công task, điều kiện cảnh báo, topic MQTT, kết quả kiểm thử và các nhóm nội dung.

Danh mục từ viết tắt

Bảng 1. Từ viết tắt dùng trong báo cáo

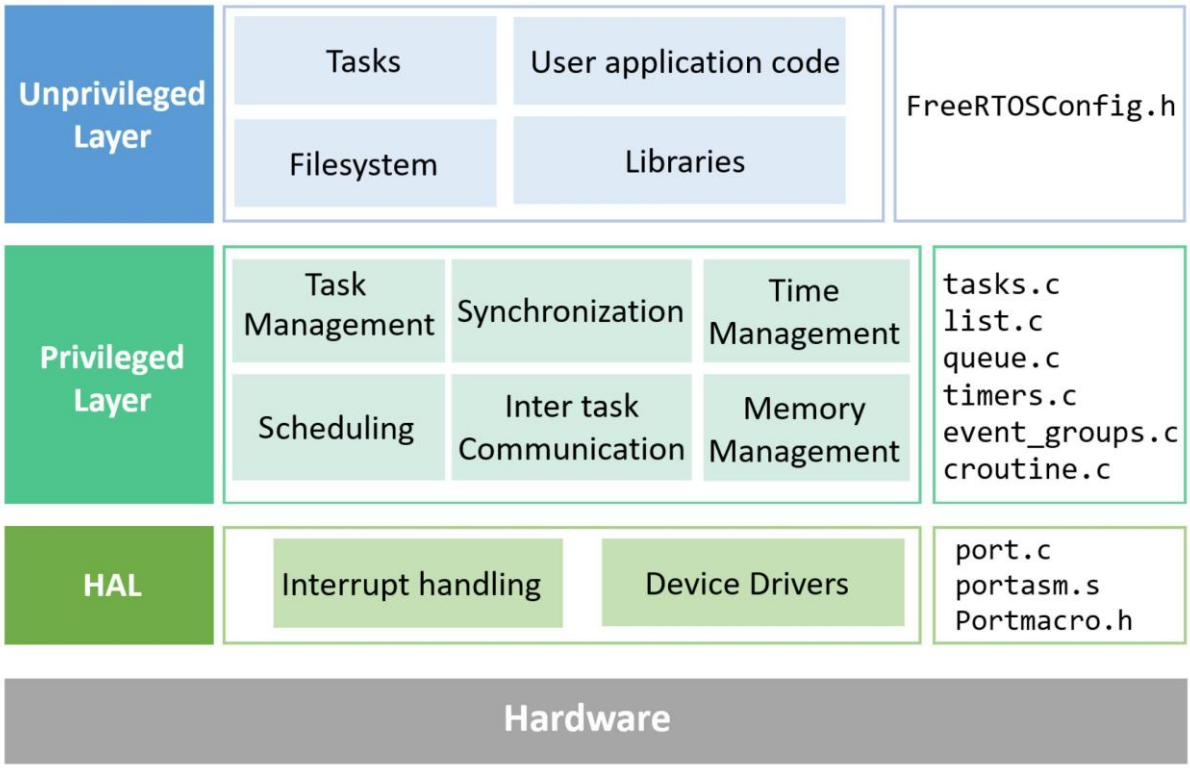
Từ	Nghĩa trong bài
RTOS	Hệ điều hành thời gian thực (Real-Time Operating System)
FreeRTOS	Nhân thời gian thực được ESP32 và Arduino-ESP32 tích hợp sẵn
Task	Tác vụ được bộ lập lịch cấp CPU
Queue	Hàng đợi truyền bản sao dữ liệu giữa các task
Mutex	Semaphore loại trừ lẫn nhau, dùng để bảo vệ vùng dùng chung
MQTT	Giao thức gửi – nhận thông điệp theo topic
BMP180	Cảm biến áp suất và nhiệt độ, giao tiếp I2C, địa chỉ 0x77 trên Wokwi
I2C	Bus hai dây SDA và SCL

GPIO	Chân vào ra số của ESP32
QoS	Mức bảo đảm gửi của MQTT. Bản này dùng QoS 0
LWT	Thông điệp broker gửi hộ khi client mất kết nối đột ngột
hPa	Hectopascal, đơn vị áp suất trên topic và trên web

Chương 1. Tổng quan về RTOS và FreeRTOS

1.1. Hệ điều hành thời gian thực

RTOS (Real-Time Operating System) là hệ điều hành thời gian thực, được thiết kế để quản lý và điều phối các tác vụ (Task) sao cho chúng được thực hiện trong khoảng thời gian xác định, đáp ứng đúng thời hạn (Deadline).



Hình 1. Kiến trúc hệ điều hành RTOS

Trong hệ thống Runner Monitoring, thời gian thực được hiểu là khả năng duy trì hoạt động đọc cảm biến và xử lý cảnh báo theo chu kỳ, ngay cả khi kết nối mạng bị chậm. Điều này không có nghĩa mọi lệnh đều phải hoàn thành trong một khoảng thời gian cố định tính bằng micro giây. Cụ thể, hệ thống được thiết lập chu kỳ đọc dữ liệu từ BMP180 là 1000 ms và chu kỳ gửi telemetry là 2000 ms. Đây là các chu kỳ được quy định trong chương trình để phục vụ hoạt động giám sát, không phải những giới hạn thời gian thực cứng của một hệ thống điều khiển công nghiệp.

Khác với hệ điều hành máy tính thông thường, vốn hướng đến khả năng xử lý nhiều chương trình và phân chia tài nguyên hợp lý, RTOS tập trung vào việc điều phối các tác vụ để chúng không cản trở lẫn nhau. Chẳng hạn, khi một task đang chờ phản hồi từ mạng, các task đọc cảm biến và xử lý cảnh báo vẫn có thể tiếp tục hoạt động. ESP32 có hai lõi xử lý và

FreeRTOS trong ESP-IDF hỗ trợ gán task vào một lõi cụ thể thông qua hàm xTaskCreatePinnedToCore().

Trong FreeRTOS, mỗi task có thể ở một trong các trạng thái: Running (đang thực thi), Ready (sẵn sàng chạy), Blocked (đang chờ) hoặc Suspended (tạm dừng). Một task chuyển sang trạng thái Blocked khi phải chờ dữ liệu từ queue, chờ tài nguyên được giải phóng hoặc chờ hết khoảng thời gian delay. Scheduler sẽ dựa vào độ ưu tiên để lựa chọn task phù hợp trong số các task đang ở trạng thái Ready.

Đối với Runner Monitoring, cả bốn task ứng dụng đều được thiết lập cùng mức ưu tiên là 2. Vì vậy, chúng không phân biệt nhau về độ ưu tiên. Việc luân phiên sử dụng CPU chủ yếu diễn ra khi các task chủ động nhường CPU thông qua vTaskDelay(), duy trì chu kỳ bằng vTaskDelayUntil() hoặc chuyển sang trạng thái chờ khi sử dụng queue.

1.2. Ưu điểm của RTOS

- **Tính xác định (determinism):** Thời gian phản hồi với sự kiện được đảm bảo trong giới hạn cho phép, đặc biệt quan trọng với hệ thống an toàn như điều khiển động cơ, y tế, ô tô.
- **Đa nhiệm (multitasking):** Chia chương trình thành nhiều Task độc lập, mỗi Task làm một việc riêng và được scheduler cấp CPU theo mức ưu tiên.
- **Lập trình theo mô-đun:** Mỗi chức năng (đọc cảm biến, giao tiếp, hiển thị...) là một Task riêng, giúp code dễ đọc, dễ bảo trì, dễ chia việc cho nhiều người.
- **Quản lý ưu tiên (priority-based scheduling):** Tác vụ quan trọng luôn được chạy trước, có thể ngắt (preempt) tác vụ ưu tiên thấp hơn.
- **Dễ mở rộng:** Thêm tính năng mới bằng cách thêm Task mà ít ảnh hưởng tới phần code cũ.
- **Cung cấp sẵn cơ chế đồng bộ và giao tiếp:** Semaphore, Mutex, Queue, Event Group, Timer phần mềm...
- **Sử dụng CPU hiệu quả:** Khi Task bị chặn (chờ sự kiện, delay), CPU được giao cho Task khác hoặc vào chế độ tiết kiệm năng lượng (idle/tickless).
- **Tái sử dụng code và dễ chuyển đổi phần cứng:** Tầng ứng dụng độc lập tương đối

1.3. FreeRTOS và ưu/nhược điểm

FreeRTOS là RTOS mã nguồn mở (giấy phép MIT), nhỏ gọn, được dùng rộng rãi trên vi điều khiển (ARM Cortex-M, ESP32, AVR, STM32...) và hiện do Amazon (AWS) duy trì.

❖ **Ưu điểm**

- Miễn phí, mã nguồn mở, dùng được cho cả sản phẩm thương mại.
- Nhỏ gọn: Kernel chỉ cần vài KB ROM và RAM, phù hợp vi điều khiển tài nguyên thấp.
- Hỗ trợ nhiều kiến trúc: Hơn 40 kiến trúc vi điều khiển, port có sẵn.
- Cộng đồng lớn, tài liệu phong phú**,** nhiều ví dụ, được tích hợp sẵn trong ESP-IDF, STM32Cube...
- Đầy đủ tính năng cơ bản: Task, Queue, Semaphore, Mutex, Software Timer, Event Group, Stream/Message Buffer, Task Notification.
- Cấu hình linh hoạt qua file FreeRTOSConfig.h (cắt bớt tính năng không cần để tiết kiệm bộ nhớ).
- Hệ sinh thái AWS IoT: Dễ kết nối đám mây (FreeRTOS libraries, OTA...).
- Có phiên bản SAFERTOS (thương mại) đạt chứng nhận an toàn chức năng.

❖ **Nhược điểm**

- Chỉ là kernel**:** Không có sẵn file system, mạng, USB, GUI... (phải dùng thêm thư viện như FreeRTOS+TCP, FatFS...).
- Không có bảo vệ bộ nhớ mặc định: Các Task dùng chung không gian địa chỉ, lỗi ở một Task (tràn stack, con trỏ sai) có thể làm hỏng cả hệ thống (chỉ có MPU port hạn chế).
- Phải tự quản lý stack cho từng Task: Cấp thiếu gây tràn stack, cấp thừa lãng phí RAM; khó debug.
- Scheduler đơn giản: Mặc định là round-robin theo ưu tiên, không có các thuật toán như EDF; dễ gặp đảo ngược ưu tiên (priority inversion) và deadlock nếu thiết kế kém.
- Hỗ trợ đa nhân (SMP) còn hạn chế so với các RTOS khác.
- Hỗ trợ chính thức hạn chế với bản miễn phí (chủ yếu qua diễn đàn); hỗ trợ thương mại phải trả phí.

- Cần hiểu biết về lập trình đồng thời: Người mới dễ mắc lỗi race condition, dùng sai API trong ISR.

Chương 2. Giao tiếp và đồng bộ giữa các task

2.1. Vì sao các task phải trao đổi dữ liệu

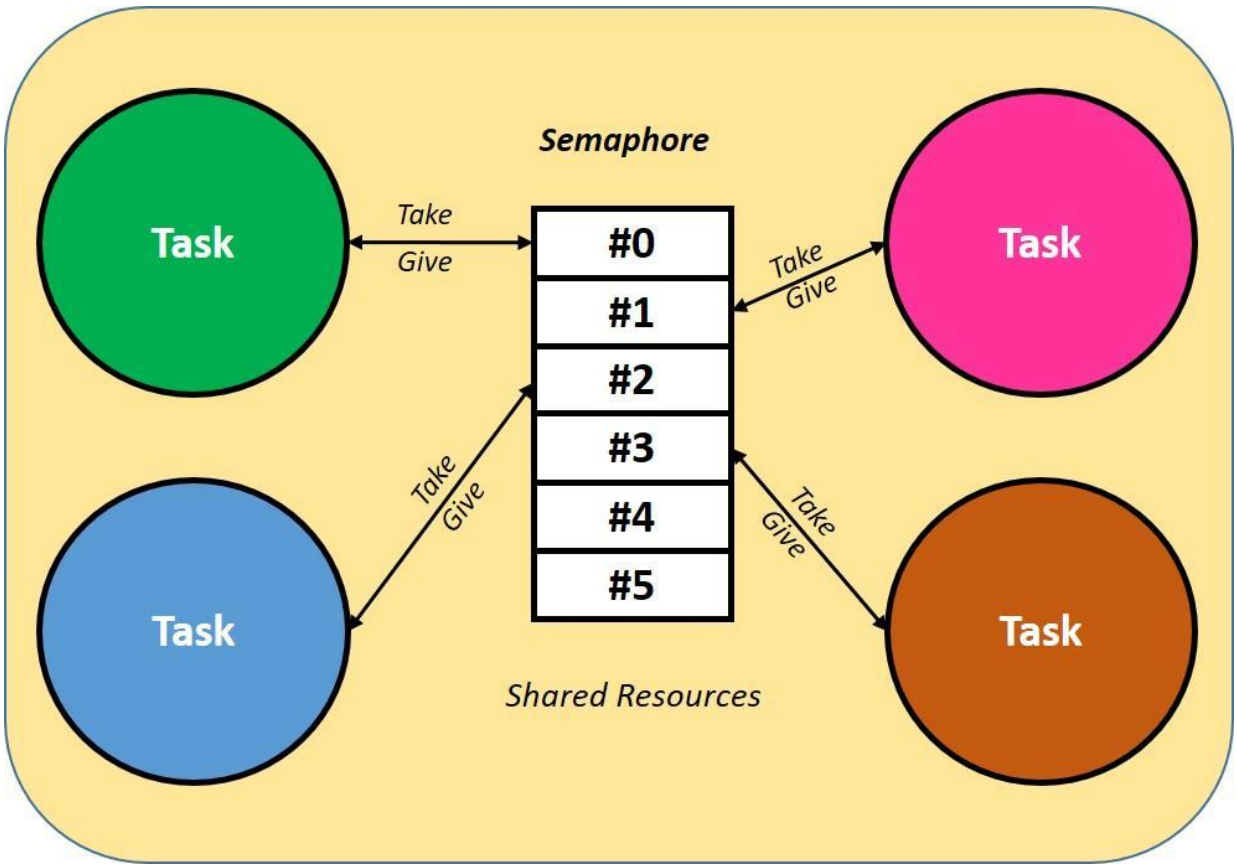
Đối với bài toán Thiết kế Mạch Giám sát và cảnh báo người chạy (runner). Task đọc cảm biến tạo ra số đo. Task mạng cần số đo đó để phát. Task cảnh báo cần nhiệt độ và độ cao để quyết định LED và còi. Nếu cả ba cùng đọc một struct toàn cục, task nhận có thể lấy nhiệt độ mới và độ cao cũ. Queue copy cả struct tại thời điểm gửi, nên bản tin là một mẫu trọn vẹn.

Đồng bộ còn để một lệnh MQTT không cắt ngang lúc task cảnh báo đang tính mức đầu ra. Mutex bảo vệ trạng thái chế độ và mức GPIO. Hàng đợi lệnh để callback của PubSubClient, vốn chạy bên trong Task_TransmitData, không tự ghi GPIO.

2.2. Cờ hiệu (Semaphore)

2.2.1. Khái niệm

Semaphore là cơ chế đồng bộ hóa trong FreeRTOS, cho phép các Task hoặc ISR thông báo cho nhau khi một sự kiện xảy ra hoặc khi tài nguyên đã sẵn sàng.



Hình 2. tổng quát cơ chế Semaphore trong RTOS

Có thể hình dung Semaphore giống như một chiếc chuông báo:

- **Give**: Phát tín hiệu.
- **Take**: Nhận tín hiệu.
- Nếu chưa có tín hiệu, Task có thể chuyển sang trạng thái Blocked để chờ.
- Khi tín hiệu được phát, Task đang chờ có thể được đưa về trạng thái Ready.

Semaphore không có nhiệm vụ truyền nội dung dữ liệu như nhiệt độ, độ cao hay trạng thái cảm biến. Nó chủ yếu thông báo rằng một sự kiện đã xảy ra.

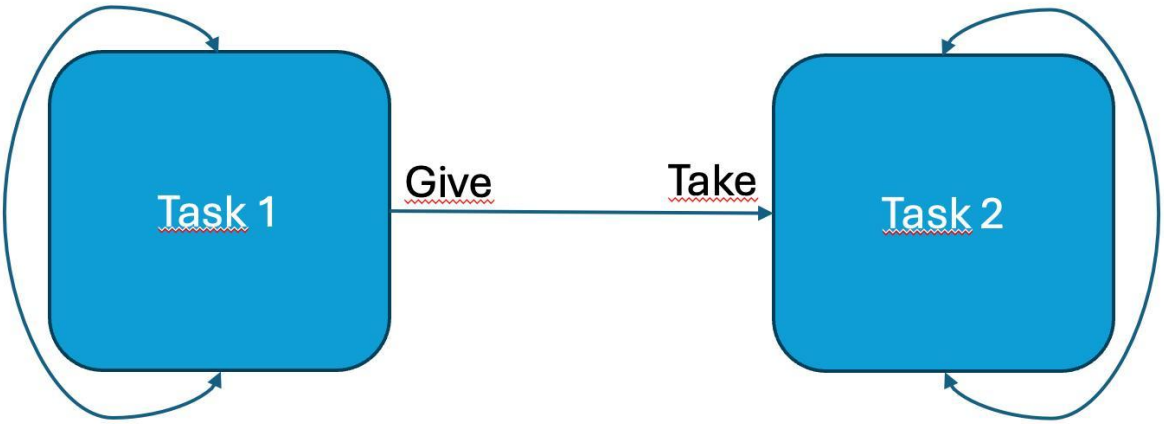
2.2.3. Các API Chính

API	Chức năng
xSemaphoreCreateBinary()	Tạo binary semaphore (count ban đầu = 0)

API	Chức năng
xSemaphoreCreateCounting(uxMax, uxInitial)	Tạo counting semaphore
xSemaphoreCreateMutex()	Tạo mutex (có kế thừa ưu tiên)
xSemaphoreTake(xSem, xTicksToWait)	Lấy semaphore, chờ tối đa số tick chỉ định
xSemaphoreGive(xSem)	Trả semaphore, đánh thức Task đang chờ
xSemaphoreGiveFromISR(xSem, &xWoken)	Give trong ISR
vSemaphoreDelete(xSem)	Xóa semaphore, giải phóng bộ nhớ

2.2.2. Nguyên lý hoạt động

1. Task 1 hoặc Interrupt phát tín hiệu (Give).
2. Semaphore chuyển sang trạng thái có tín hiệu.
3. Task 2 đang chờ được đánh thức (Take).
4. Task 2 tiếp tục thực thi công việc.



Ví dụ: Trong hệ thống Runner Monitoring:

1. Task đọc BMP180 hoàn thành việc lấy dữ liệu.
2. Task này gọi `xSemaphoreGive()` để phát tín hiệu.
3. Task xử lý cảnh báo đang chờ bằng `xSemaphoreTake()`.
4. Khi nhận được tín hiệu, Task xử lý cảnh báo được phép tiếp tục thực hiện.

Nhờ đó, Task xử lý không cần liên tục kiểm tra xem Task đọc cảm biến đã hoàn thành hay chưa.

2.2.3. Các loại Semaphore

a) Binary Semaphore

- Giá trị chỉ 0 hoặc 1**,** giống một "cờ" bật/tắt.

- Dùng để đồng bộ (báo sự kiện)**:** một bên Give, bên kia Take.
- Thường không có kế thừa ưu tiên**,** nên không phù hợp để khóa tài nguyên.

b) Counting Semaphore

- Giá trị từ 0 đến N
- Hai cách dùng chính:
    1. Đếm sự kiện: ISR Give nhiều lần, Task Take từng lần một, nhờ **đó** không bị mất sự kiện khi sự kiện đến dồn dập (binary semaphore sẽ mất vì chỉ nhớ được 1).
    2. Quản lý nhóm tài nguyên: khởi tạo bằng số tài nguyên có sẵn (ví dụ 3 kết nối), mỗi Task dùng thì Take, xong thì Give.

c) Mutex (Mutual Exclusion)

- Là binary semaphore có thêm kế thừa ưu tiên (priority inheritance)**.**
- Quy tắc sở hữu: Task nào Take thì chính Task đó phải Give.
- Không dùng trong ISR.
- Dùng để loại trừ tương hỗ**:** bảo đảm tại một thời điểm chỉ một Task dùng tài nguyên.

d) Recursive Mutex

- Cùng một Task có thể Take nhiều lần (phải Give đúng số lần đó).
- Dùng khi hàm A giữ khóa và gọi hàm B cũng cần khóa đó.
- API: xSemaphoreCreateRecursiveMutex(), xSemaphoreTakeRecursive(), xSemaphoreGiveRecursive().

2.2.5. Ưu điểm của semaphore

Ưu điểm	Ý nghĩa
Nhẹ	Chỉ cần một vài chục byte RAM, thao tác rất nhanh
Tiết kiệm CPU	Task ngủ chờ thay vì polling liên tục
Đồng bộ ISR ↔ Task	Cho phép "hoãn xử lý ngắt" (deferred interrupt processing): ISR ngắn gọn, việc nặng giao cho Task
Chống xung đột tài nguyên	Mutex đảm bảo loại trừ tương hỗ
Chống đảo ngược ưu tiên	Nhờ priority inheritance của mutex
Có timeout	Tránh Task bị treo vô hạn
Quản lý được số lượng	Counting semaphore giới hạn số tài nguyên/kết nối

Hạn chế : không truyền dữ liệu; dễ gây deadlock (hai Task giữ khóa chờ nhau) nếu lấy nhiều mutex không theo thứ tự cố định; quên Give sẽ khiến các Task khác bị treo.

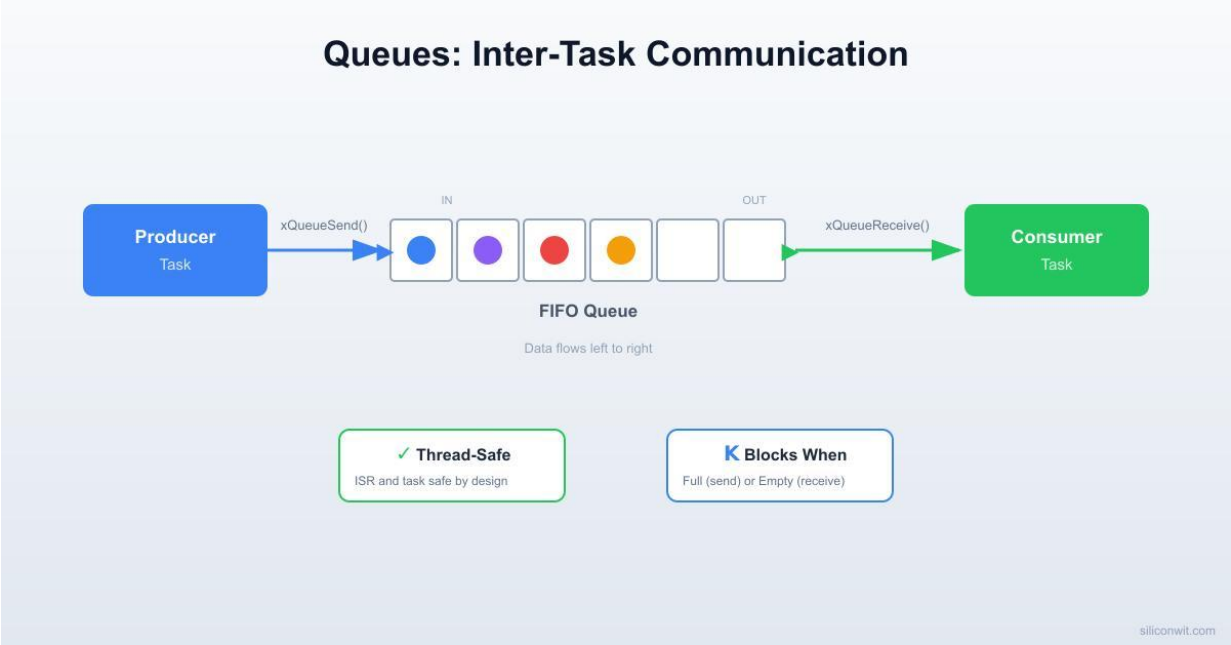
2.2.6. Ứng dụng thực tế của Semaphore:

- Đồng bộ hoạt động giữa các Task trong hệ thống.
- Thông báo khi cảm biến đọc dữ liệu xong.
- Thông báo khi xảy ra sự kiện từ ngắt (ISR).
- Báo hiệu khi quá trình truyền UART, SPI, I2C hoàn tất.
- Đồng bộ giữa Task và DMA khi truyền hoặc nhận dữ liệu.
- Quản lý quyền truy cập tài nguyên dùng chung như UART, I2C, SPI.
- Quản lý số lượng tài nguyên có giới hạn trong hệ thống.
- Đồng bộ các Task trong hệ thống điều khiển và giám sát thời gian thực.

2.3. Hàng đợi (Queue)

2.3.1. Khái niệm

Queue là cơ chế trao đổi dữ liệu giữa các Task thông qua một vùng bộ nhớ đệm. Dữ liệu được gửi vào và lấy ra theo nguyên tắc FIFO (First In – First Out).



Hình 3. tổng quát cơ chế Queue trong RTOS

Các đặc điểm quan trọng:

- **Truyền bằng cách sao chép (copy by value):** dữ liệu được copy vào queue, nên bên gửi có thể dùng lại/xóa biến gốc ngay sau khi gửi (an toàn, không sợ biến cục bộ bị hủy).
- **Blocking có timeout ở cả hai phía:**
  - Nhận khi queue **rỗng** → Task bị Blocked chờ dữ liệu.
  - Gửi khi queue **đầy** → Task bị Blocked chờ có chỗ trống.
- **Nhiều Task có thể cùng gửi và cùng nhận.** Nếu nhiều Task cùng chờ nhận, Task có ưu tiên cao nhất (hoặc chờ lâu nhất nếu bằng ưu tiên) được nhận trước.
- Phần tử có thể là **số, struct, con trỏ** đều được.

2.3.2 Các API chính

API	Chức năng
xQueueCreate(len, size)	Tạo queue
xQueueSend() / xQueueSendToBack()	Gửi vào cuối hàng
xQueueSendToFront()	Gửi vào đầu hàng (dữ liệu khẩn cấp)
xQueueReceive()	Nhận và **lấy ra** khỏi queue
xQueuePeek()	Đọc **không lấy ra**
xQueueOverwrite()	Ghi đè (cho queue 1 phần tử, kiểu "mailbox")
uxQueueMessagesWaiting()	Số phần tử đang chứa
xQueueSendFromISR() / xQueueReceiveFromISR()	Phiên bản dùng trong ngắt

2.3.2. Nguyên lý hoạt động

1. Task A tạo dữ liệu cảm biến.
2. Task A gửi dữ liệu vào Queue (Send).
3. Queue lưu trữ dữ liệu trong bộ nhớ đệm.
4. Task B nhận dữ liệu từ Queue (Receive).
5. Task B xử lý dữ liệu nhận được.

2.3.3 Queue Set và Mailbox

- **Queue Set:** cho một Task chờ nhiều queue/semaphore cùng lúc (xQueueCreateSet, xQueueSelectFromSet). Với thiết kế mới, thường thay bằng Event Group hoặc Task Notification cho gọn.
- **Mailbox (queue 1 phần tử + xQueueOverwrite):** luôn giữ giá trị mới nhất, phù hợp trạng thái cần "giá trị hiện tại" như nhiệt độ, tốc độ động cơ.

2.3.3. Ưu điểm

- Trao đổi dữ liệu giữa nhiều Task.
- Lưu trữ dữ liệu tạm thời khi Task nhận chưa xử lý kịp.
- Giảm sự phụ thuộc trực tiếp giữa các Task.
- Hỗ trợ cơ chế FIFO, đảm bảo thứ tự dữ liệu.
- Hỗ trợ truyền dữ liệu an toàn giữa các Task.
- Có thể gửi dữ liệu từ Interrupt vào Queue bằng API chuyên dụng.

2.3.4. Ứng dụng

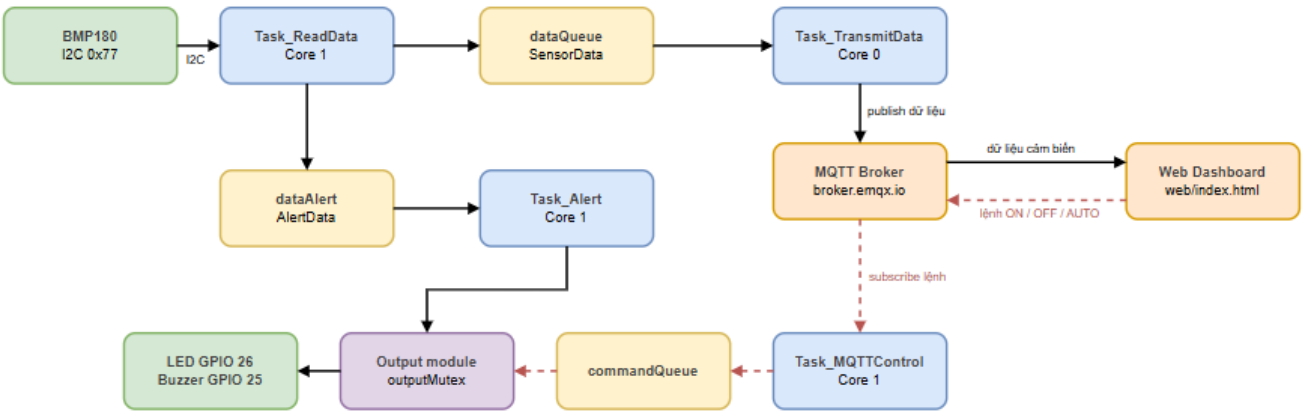
- Truyền dữ liệu cảm biến giữa Task đọc và Task xử lý.
- Gửi dữ liệu UART đến Task giao tiếp.
- Lưu trữ các sự kiện chờ xử lý.
- Truyền dữ liệu giữa các Task điều khiển và giám sát.
- Xây dựng hệ thống Producer – Consumer.

Chương 3. Thiết kế và xây dựng hệ thống

3.1. Tổng quan

Mục tiêu là giám sát nhiệt độ môi trường, áp suất khí quyển và độ cao ước tính, rồi cảnh báo bằng LED và còi. Hệ thống không đo thân nhiệt và không đánh giá sức khỏe người chạy. Độ cao là kết quả của công thức khí áp so với 101325 Pa, không phải số đo GPS.

Phần cứng trên Wokwi gồm board ESP32 DevKit C V4, BMP180, điện trở 1 kΩ, LED đỏ và buzzer piezo. Phần mềm gồm firmware Arduino-ESP32, thư viện Adafruit BMP085 1.2.4 để đọc BMP180, PubSubClient 2.8, và trang web tĩnh trong thư mục web. Broker là broker.emqx.io.



Hình 4. Kiến trúc các task, queue, broker và web.

3.2. Phần cứng và sơ đồ Wokwi

3.2.1. Danh sách linh kiện phần cứng và sơ đồ chân kết nối

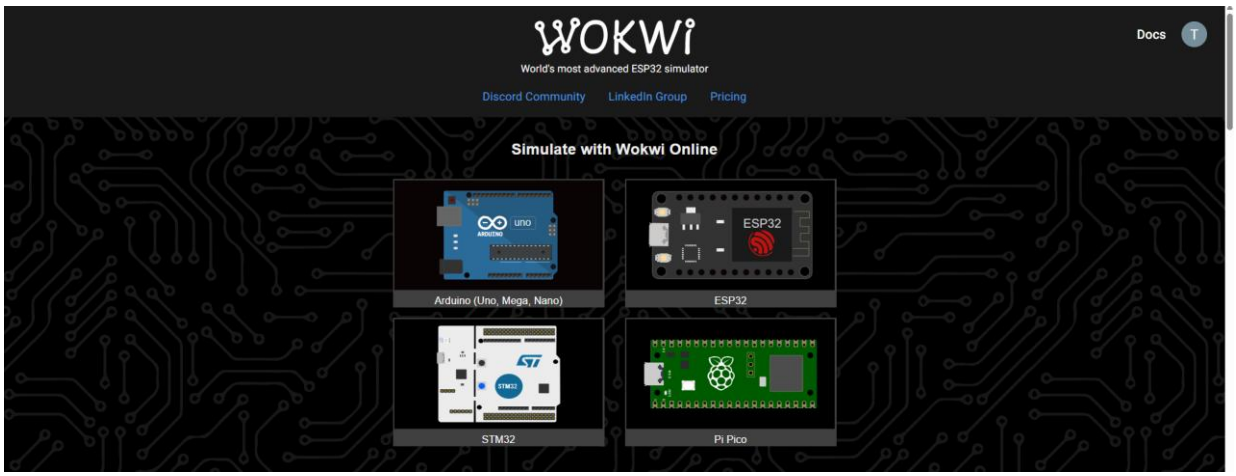
Link kiện	**Chân thiết bị**	**Điểm ESP32**	**Chức năng**
BMP180	VCC	3V3	Nguồn
BMP180	GND	GND.1	Mass
BMP180	SDA	GPIO 21	Dữ liệu I2C
BMP180	SCL	GPIO 22	Xung I2C
Điện trở 1 kΩ	Một đầu của led	GPIO 26	Hạn dòng LED
LED đỏ	Anode	Đầu còn lại của điện trở	Sáng khi GPIO 26 ở mức cao
LED đỏ	Cathode	GND.3	Mass
Buzzer	Pin 2, cực dương	GPIO 25	tone 2000 Hz khi bật
Buzzer	Pin 1, cực âm	GND.3	Mass
Serial	TX và RX của ESP32	Serial Monitor Wokwi	Log 115200 baud

Bảng danh sách linh kiện sử dụng và cấu hình chân

3.2.2. Sơ đồ kết nối phần cứng trên Wokwi

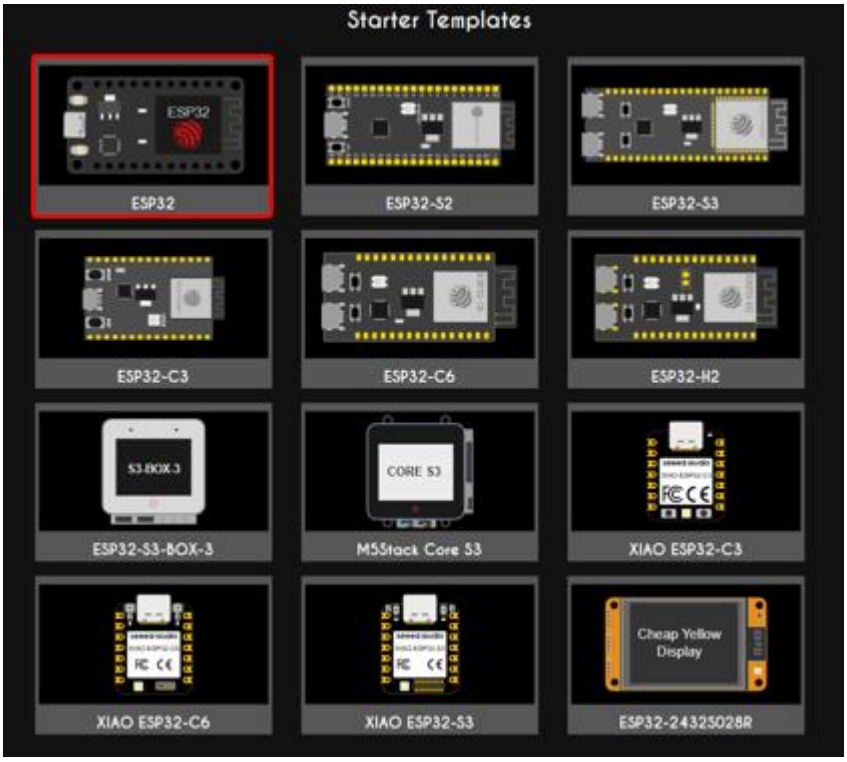
3.2.2.1. thiết lập sơ đồ kết nối chân

Bước 1 : Vào https://wokwi.com/ đăng nhập tài khoản

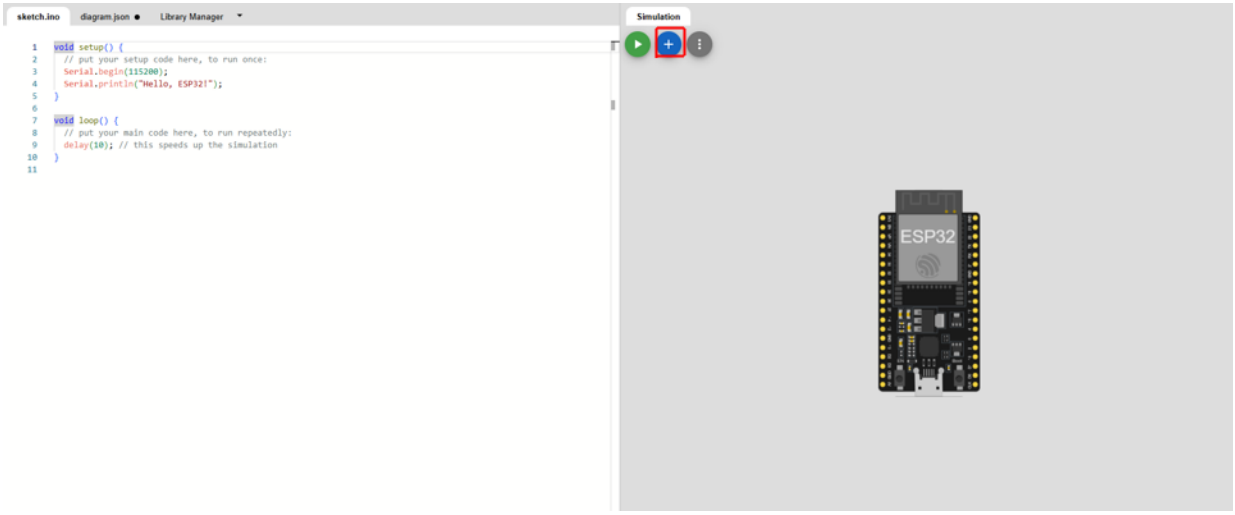


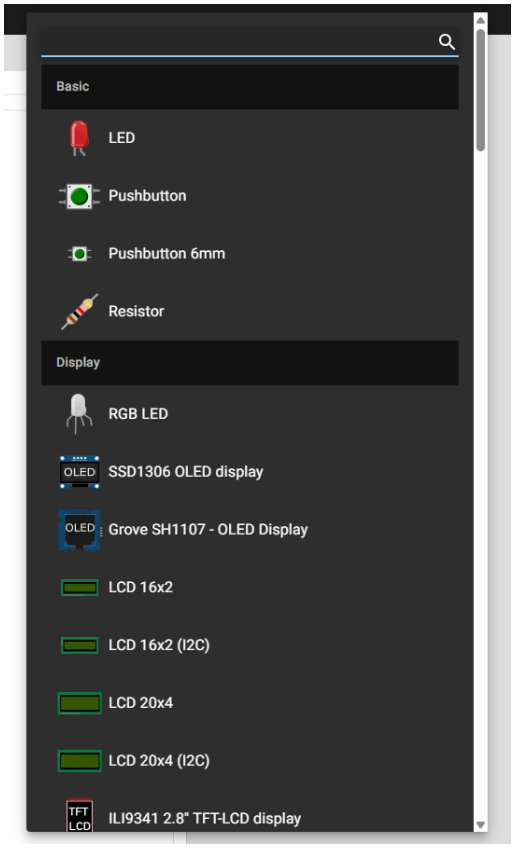
Bước 2 : Chọn ESP32 sau đó chọn phiên bản là ESP32-DEVKIT



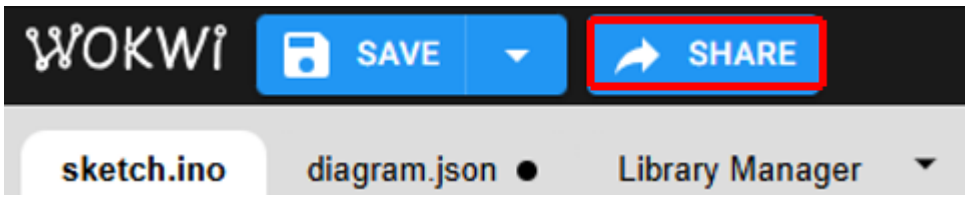


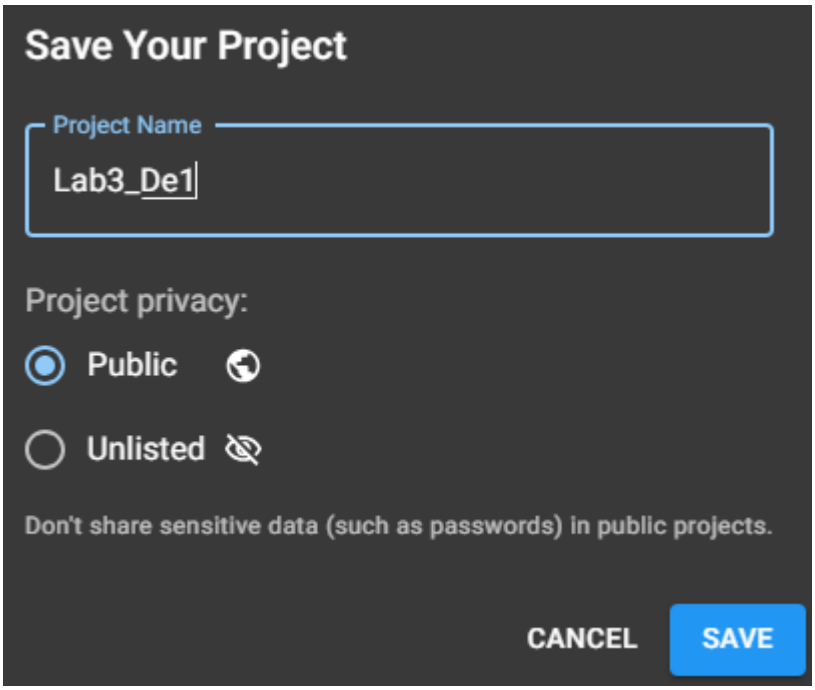
Bước 3 chọn dấu + để và nhập tên để thêm linh kiên





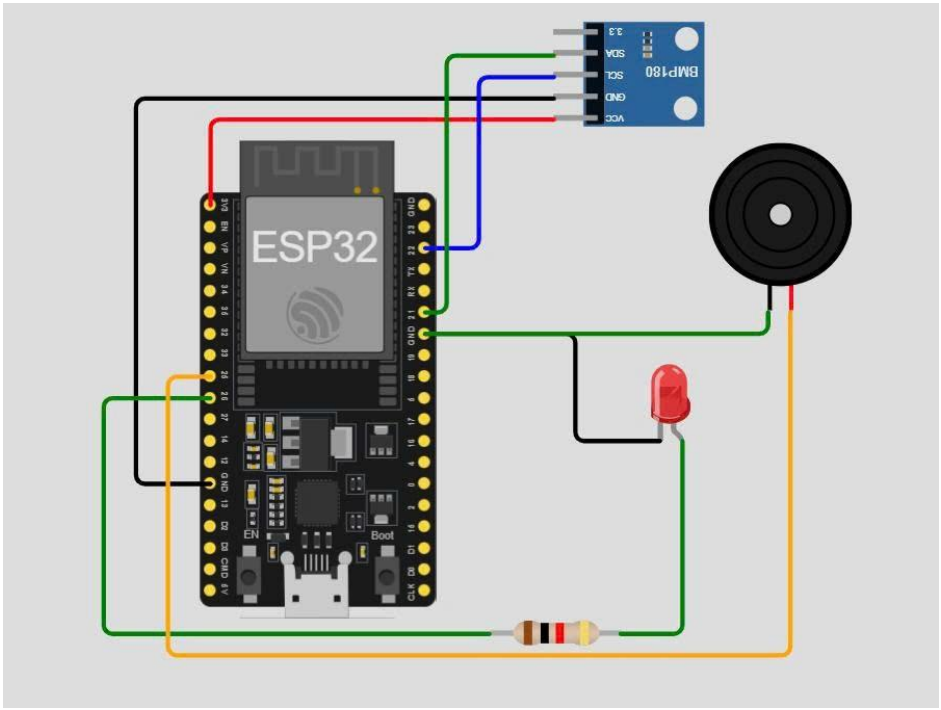
Bướ 4 : Sau khi hoàn thành để share ấn share rồi save project điên tên và ấn save





❖ **Sơ đồ mạch hoàn chỉnh trong wokwi**

Link share : https://wokwi.com/projects/480532359100668929



3.3. BMP180 và cách tính độ cao

BMP180 là cảm biến áp suất khí quyển và nhiệt độ kỹ thuật số do Bosch phát triển.

Cảm biến sử dụng áp suất khí quyển để ước tính độ cao so với mực nước biển.

BMP180 thường được sử dụng trong:

- Thiết bị đo độ cao (Altimeter).
- Trạm quan trắc thời tiết.
- Máy bay mô hình, drone.
- Robot và hệ thống định vị.
- Thiết bị IoT giám sát môi trường.

❖ Các thông số chính

Thông số	Giá trị
Điện áp hoạt động	1.8–3.6 V
Giao tiếp	I2C
Địa chỉ I2C	0x77
Đo áp suất	300–1100 hPa
Đo nhiệt độ	-40 đến 85°C
Độ phân giải áp suất	Tùy chế độ OSS
Đo độ cao	Ước tính từ áp suất

❖ Nguyên lý hoạt động

BMP180 không đo độ cao trực tiếp. Nó đo áp suất khí quyển, sau đó vi điều khiển sử dụng công thức toán học để suy ra độ cao.

Nguyên lý:

1. Cảm biến tiếp nhận áp suất không khí xung quanh.
2. BMP180 chuyển đổi áp suất thành dữ liệu số.
3. Vi điều khiển đọc dữ liệu qua giao tiếp I2C.
4. Sử dụng công thức khí quyển để tính độ cao.

+) Mối quan hệ giữa áp suất và độ cao:

- Càng lên cao, áp suất khí quyển càng giảm.
- Càng xuống thấp, áp suất khí quyển càng tăng.

❖ **Công thức tính độ cao**

❖ Công thức thường được sử dụng :

$$h = 44330 \left[1 - \left(\frac{P}{P_0}\right)^{0.1903}\right]$$

Bảng giải thích :

Ký hiệu	**Ý nghĩa**	**Đơn vị**
h	Độ cao ước tính	m
P	Áp suất đo được	hPa
P_0	Áp suất tại mực nước biển	hPa
44330	Hằng số mô hình khí quyển	m
0.1903	Số mũ trong mô hình	Không có

- Ở trong bài tập đang thiết lập $P_0 =$ 101325

Code thiết lập công thức tính độ cao trong bài :

```
altitude = 44330 * (1 - (pressurePa / 101325) ^ 0.1903)

pressure_hPa = pressurePa / 100
```

3.4. Các Task được sử dụng

Bảng 6. Bảng dữ liệu cho các Task

Task	Core	Priority	Stack	Chức năng
Task_ReadData	1	2	6144 byte	Đọc BMP180 mỗi 1000 ms (nhiệt độ, áp suất, cao độ), kiểm tra mẫu hợp lệ, gửi mẫu mới nhất vào dataQueue và dataAlert.
Task_TransmitData	0	2	10240 byte	Duy trì Wi-Fi và MQTT, nhận lệnh điều khiển rồi đưa vào commandQueue, xuất bản telemetry từ dataQueue (chu kỳ 2000 ms) và trạng thái từ statusMailbox.
Task_Alert	1	2	5120 byte	Nhận dataAlert, phân loại cảnh báo nhiệt độ và biến thiên cao độ trong cửa sổ khoảng 5 giây, cập nhật LED và còi.
Task_MQTTControl	1	2	4096 byte	Chờ lệnh trong commandQueue và áp dụng ON, OFF hoặc AUTO cho LED hoặc còi.

Task_ReadData dùng vTaskDelayUntil để giữ chu kỳ 1000 ms tính từ mốc thức dậy, không cộng dồn thời gian đọc I2C vào chu kỳ. Task in một dòng Serial dạng T, P, Alt và timestamp. Task này không gọi Wi-Fi hay MQTT.

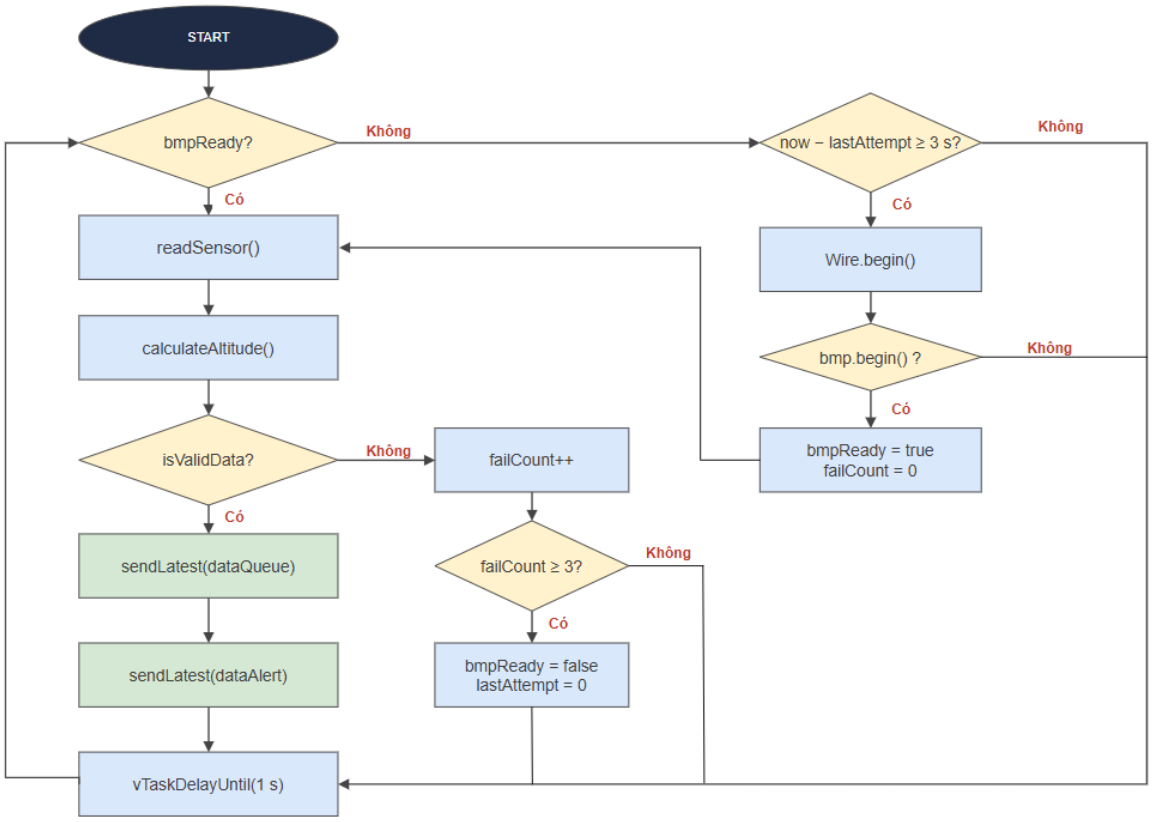
Task_TransmitData gọi WiFi.mode(WIFI_STA), WiFi.setSleep(false) và WiFi.begin với SSID Wokwi-GUEST, mật khẩu rỗng, kênh 6. Mất Wi-Fi thì thử lại sau 10000 ms. Khi có Wi-Fi mà MQTT chưa nối, task thử lại sau 4000 ms. Client ID có dạng esp32-lab2- theo millis. Keepalive MQTT là 120 giây và socket timeout là 2 giây, chọn như vậy vì Wokwi có thể chạy chậm hơn thời gian thật. Vòng lặp của task kết thúc bằng vTaskDelay(20 ms).

Task_Alert chờ dataAlert tối đa 500 ms. Khi có mẫu, task xử lý hết các mẫu đang nằm trong hàng để không bỏ lịch sử độ cao. Nhiệt độ được phân loại ngay. Độ cao đi qua bộ theo dõi 5 giây. Kết quả đưa vào outputApplyAlert.

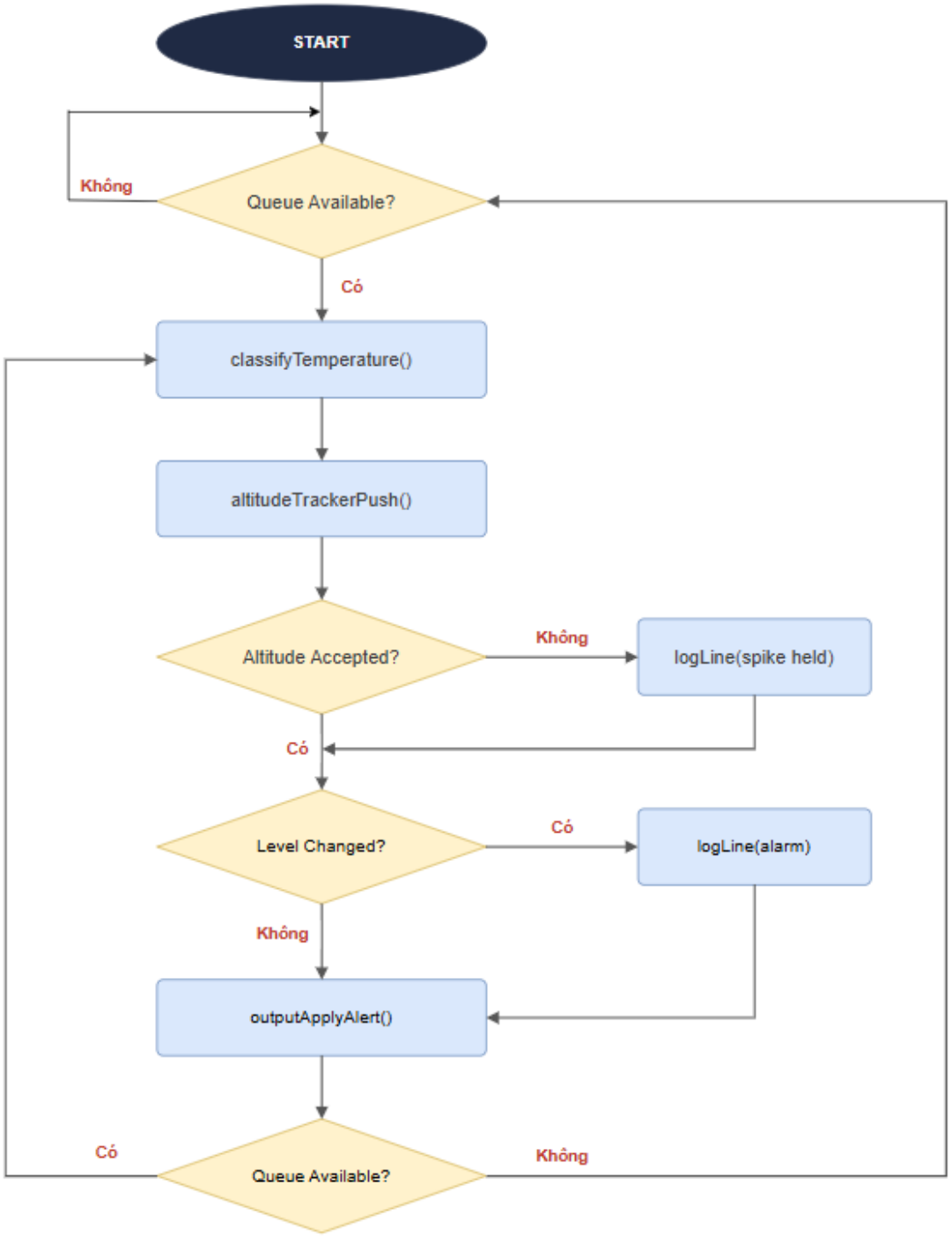
Task_MQTTControl chờ commandQueue bằng portMAX_DELAY vì task này không có việc nào khác. Mỗi lệnh chỉ sửa đúng một thiết bị. Lệnh AUTO áp dụng ngay mức cảnh báo đang lưu.

3.5. Lưu đồ thuật toán

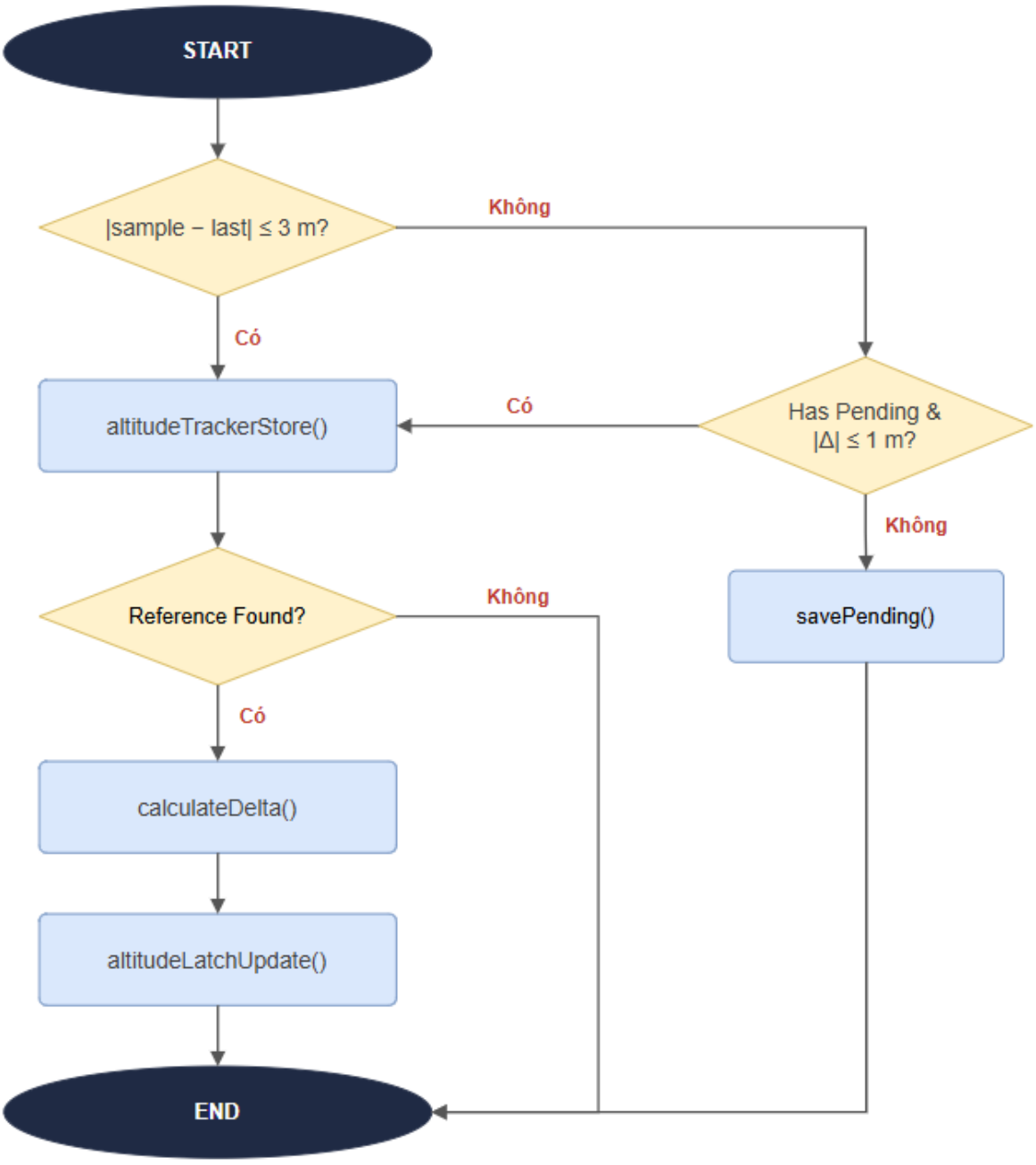
❖ Task_ReadData



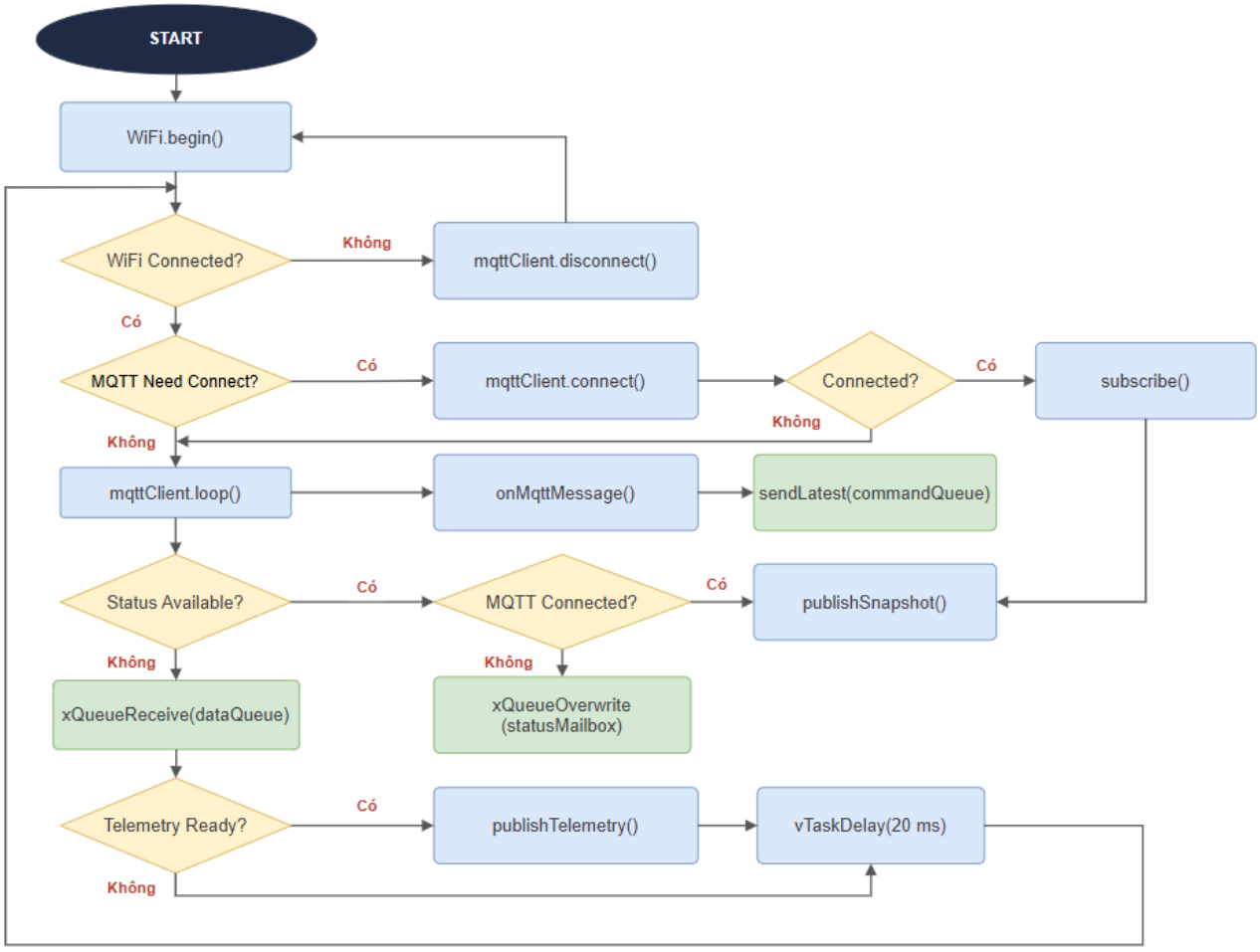
❖ **Task_Alert**


START
Queue Available?
Không
Có
classifyTemperature()
altitudeTrackerPush()
Altitude Accepted?
Không
logLine(spike held)
Có
Level Changed?
Có
logLine(alarm)
Không
outputApplyAlert()
Queue Available?
Có
Không

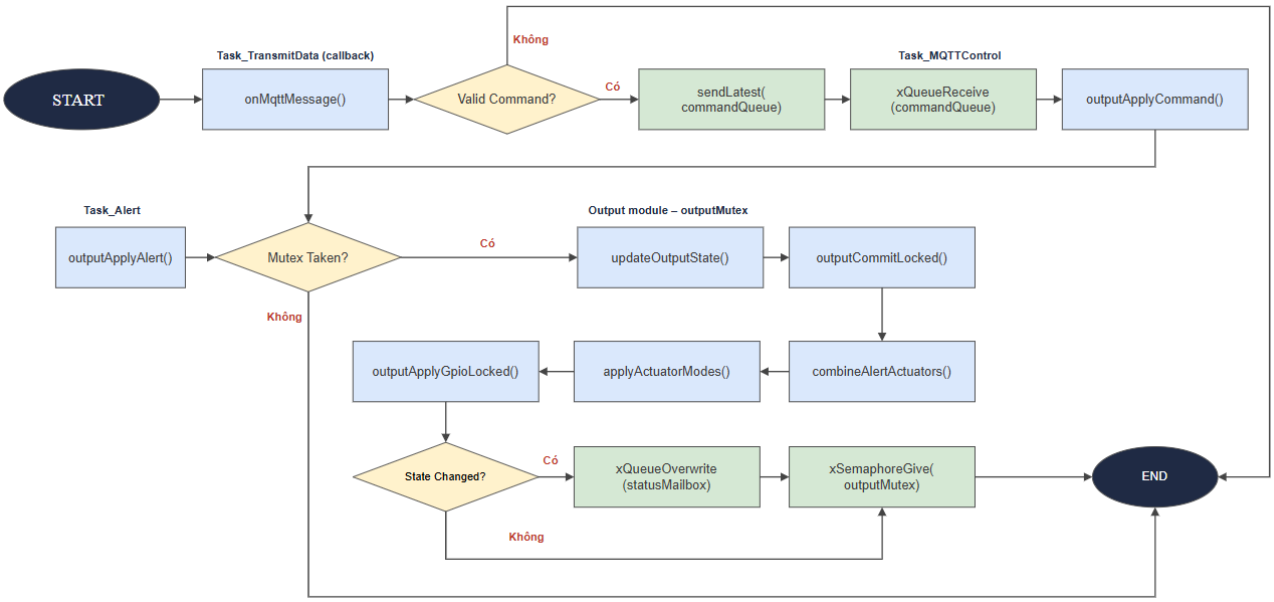
❖ **altitudeTrackerPush()**



❖ Task_TransmitData



❖ **Task_MQTTControl**



3.6. Thiết lập điều kiện cảnh báo và chế độ MANUAL/AUTO

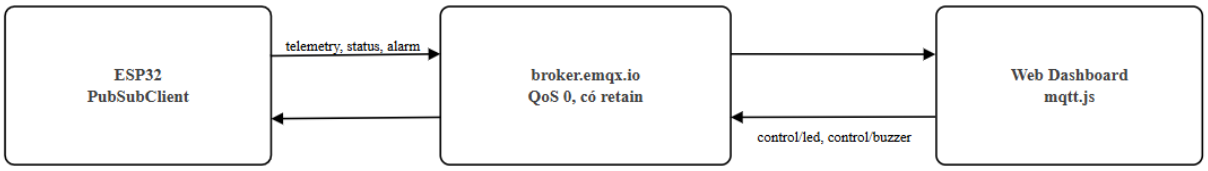
Bảng 7. Điều kiện cảnh báo cho còi và led

Điều kiện	LED	Còi
Nhiệt độ từ 10 °C đến 32 °C và không có cảnh báo độ cao	Tắt	Tắt
Lớn hơn 32 °C và nhỏ hơn hoặc bằng 35 °C	Bật	Tắt
Lớn hơn 35 °C hoặc Nhỏ hơn 10 °C	Bật	Kêu
Độ cao đổi từ 5 m đến dưới 10 m trong khoảng 5 giây	Bật	Tắt
Độ cao đổi từ 10 m trở lên trong khoảng 5 giây	Bật	Kêu

Chương 4. MQTT và giao diện web

4.1. MQTT trong hệ thống

MQTT là giao thức xuất bản và đăng ký. Client gửi một thông điệp lên một topic. Broker chuyển thông điệp tới client nào đã subscribe topic đó. ESP32 là publisher của số đo, trạng thái và cảnh báo, đồng thời là subscriber của hai topic lệnh. Web là subscriber của số đo và là publisher của lệnh. Broker trong bài là broker công khai broker.emqx.io. QoS trên cả PubSubClient và mqtt.js của trang là 0: gửi một lần, không có bảo đảm tay ba. Một số bản tin được retain để trang vừa mở vẫn nhận được giá trị mới nhất.



Hình 5. Kiến trúc truyền thông MQTT của hệ thống

4.2. Wi-Fi, broker và topic

❖ **Wi-Fi** là phương thức kết nối không dây giúp ESP32 truy cập vào mạng Internet hoặc mạng LAN.

Vai trò:

- Kết nối ESP32 với Router Wi-Fi.
- Cung cấp đường truyền để gửi và nhận dữ liệu.
- Cho phép ESP32 giao tiếp với MQTT Broker.

❖ **Broker** là máy chủ trung gian tiếp nhận, phân phối và quản lý các thông điệp MQTT giữa các thiết bị. Broker hoạt động theo mô hình Publish/Subscribe (Xuất bản/Đăng ký).

+) Có hai cơ chế chính:

- **Publish:** Thiết bị gửi dữ liệu lên Broker.
- **Subscribe:** Thiết bị đăng ký nhận dữ liệu từ một Topic.

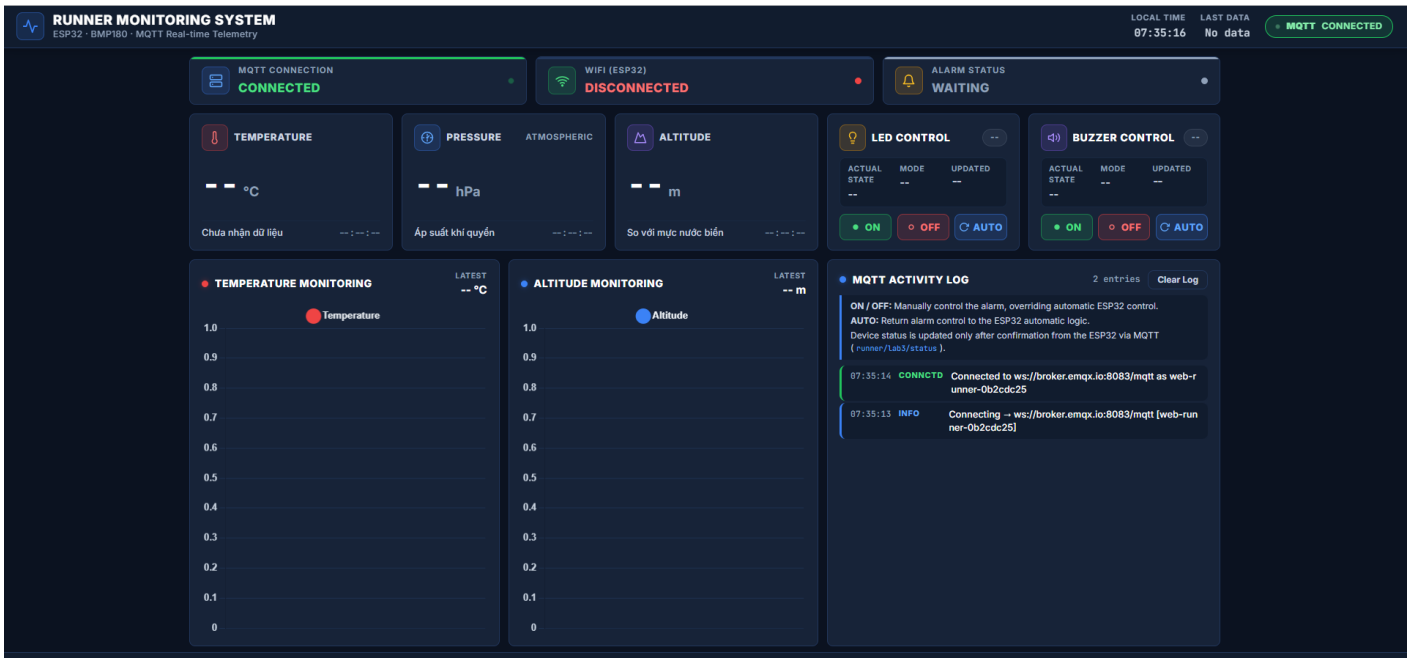
❖ **Topic** là địa chỉ phân loại thông điệp MQTT, giúp Broker xác định thông điệp cần được chuyển đến những thiết bị nào đã đăng ký nhận.

Bảng 8. Topic dùng chung bởi firmware/esp32 và web/app.js

Topic	Chiều	Payload
runner/lab3/temperature	ESP32 phát, web nhận	Số °C, ví dụ 28.00
runner/lab3/pressure	ESP32 phát, web nhận	Số hPa, ví dụ 1013.25
runner/lab3/altitude	ESP32 phát, web nhận	Số mét
runner/lab3/wifi	ESP32 và LWT	CONNECTED hoặc DISCONNECTED
runner/lab3/alarm/temperature	ESP32 phát, web nhận	HIGH, LOW hoặc NORMAL
runner/lab3/alarm/altitude	ESP32 phát, web nhận	SUDDEN_CHANGE hoặc CLEAR
runner/lab3/status	ESP32 phát, web nhận	Chuỗi key=value cách nhau bởi dấu chấm phẩy
runner/lab3/control/led	Web phát, ESP32 nhận	ON, OFF hoặc AUTO
runner/lab3/control/buzzer	Web phát, ESP32 nhận	ON, OFF hoặc AUTO

4.4. Giao diện web (index.html, style.css và app.js)

Link web : https://nttrungdtvt1.github.io/Lab3_He_Thong_Nhung_msv231432378/



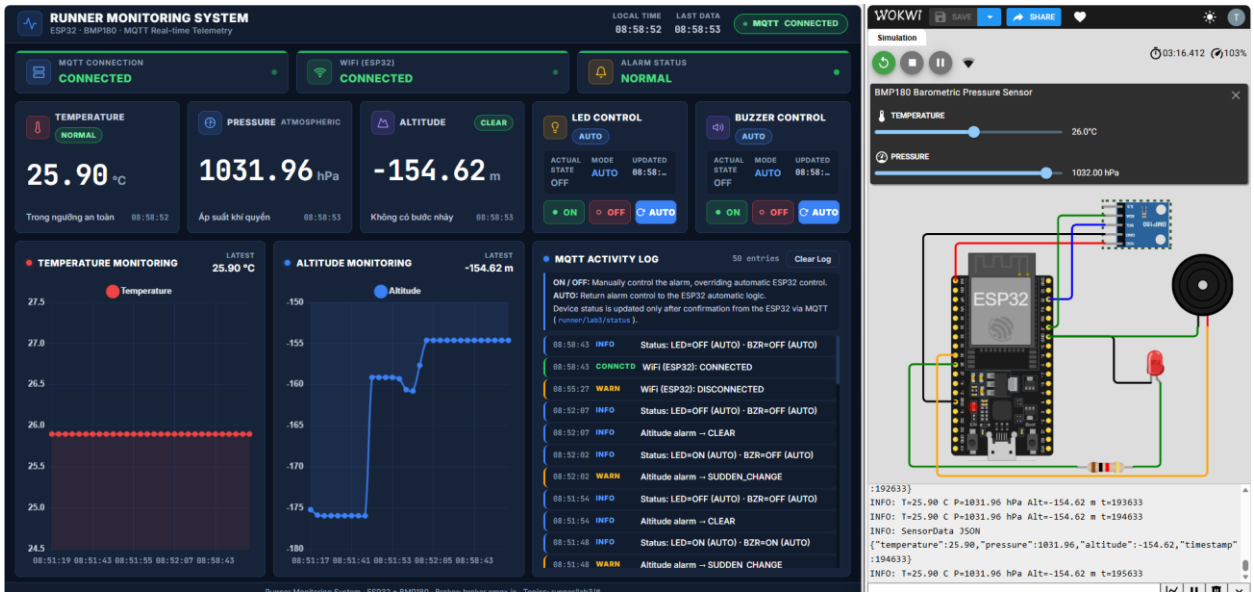
Hình 6. Dashboard của RUNNER MONITORING SYSTEM.

4.5. Code chương trình (Lab3_de1.ino và alert_logic.h)

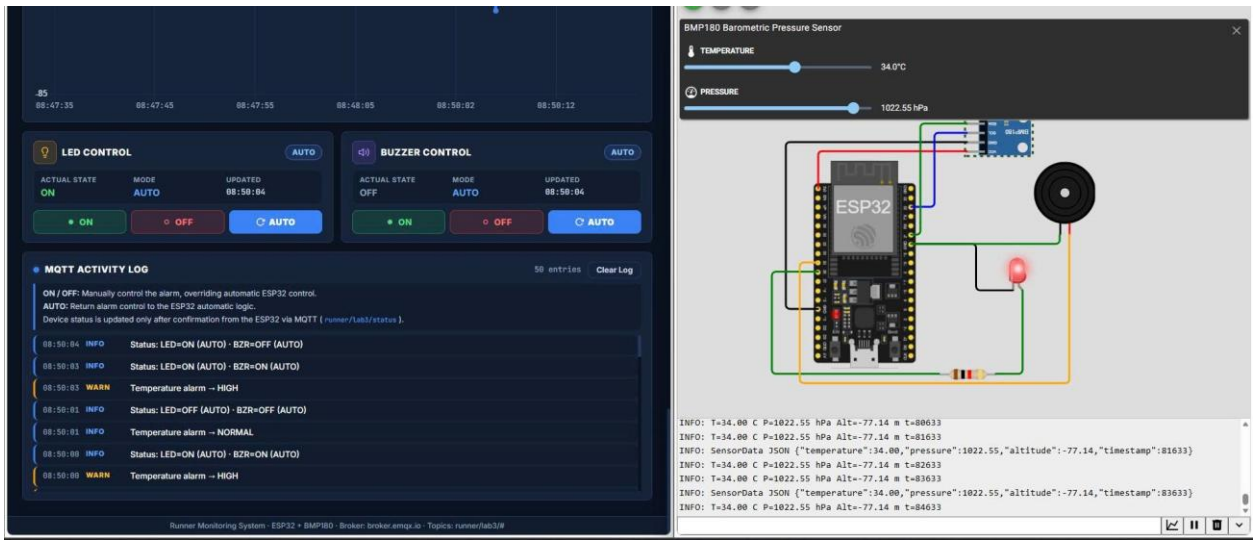
Nằm ở github (Gồm code và giải thích chi tiết):

https://github.com/nttrungdtvt1/Lab3_He_Thong_Nhung_msv231432378.git

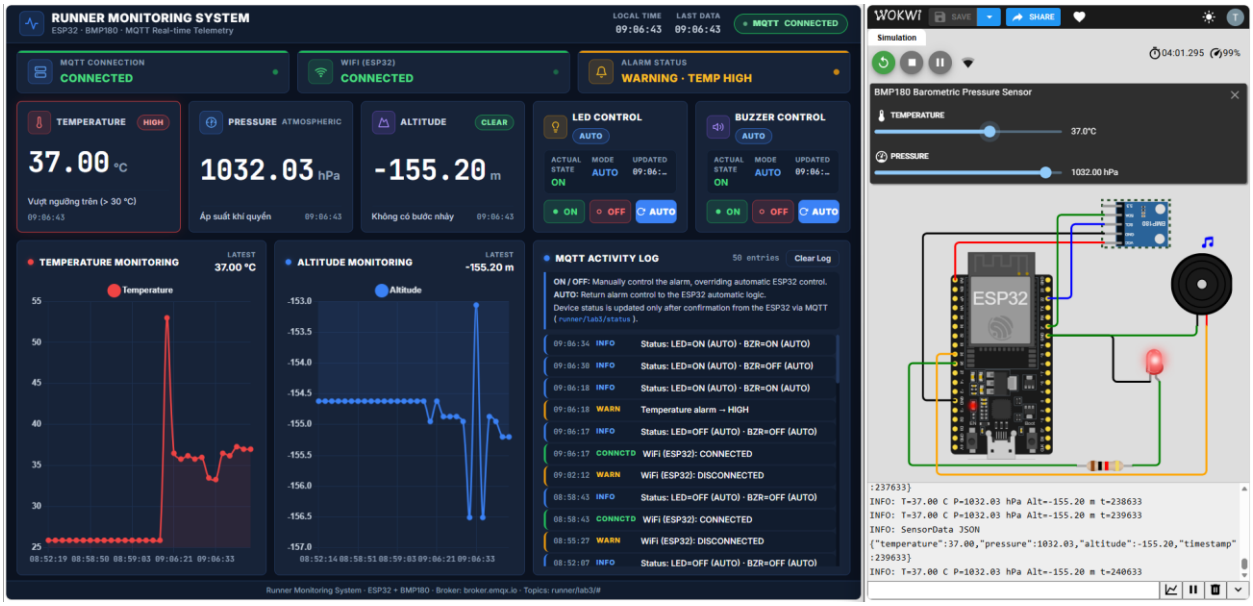
Chương 5. Kết quả mô phỏng



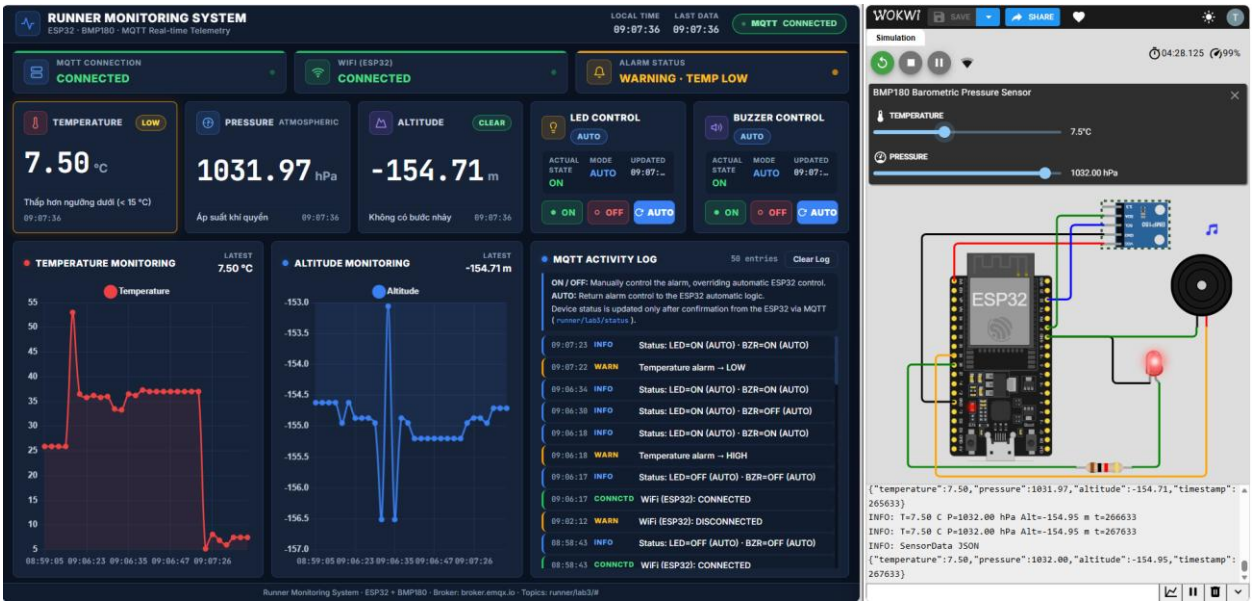
Hình 7. Nhiệt độ từ 10 °C đến 32 °C và độ cao không đổi (Led (off), còi (off)).



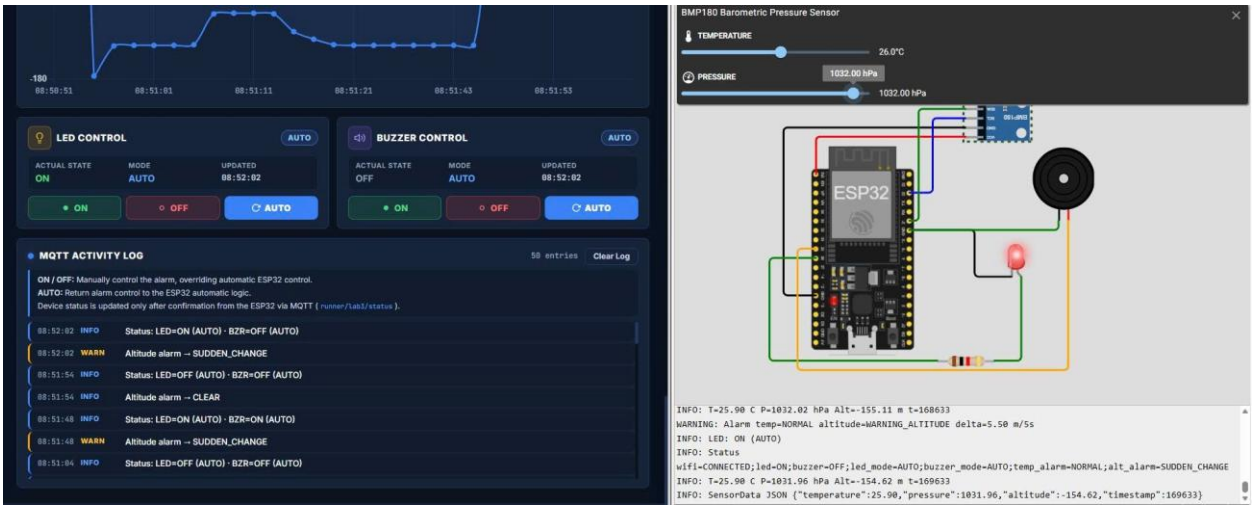
Hình 8. Nhiệt độ Lớn hơn 32 °C và nhỏ hơn hoặc bằng 35 °C và độ cao không đổi (Led (on), còi (off)).



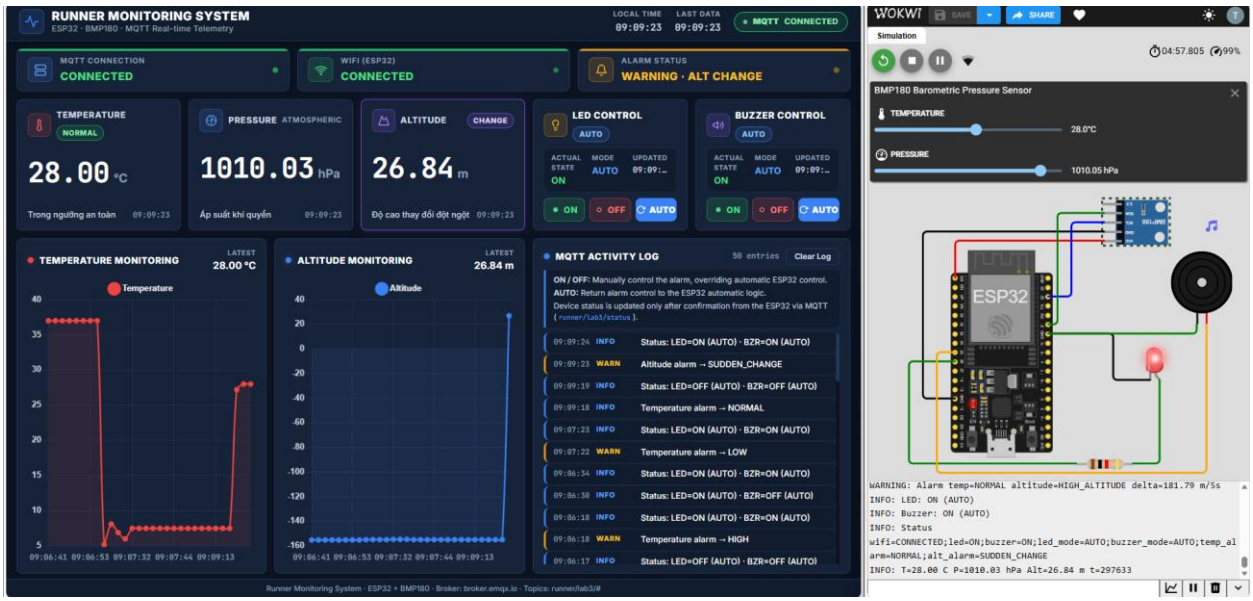
Hình 9. Nhiệt độ lớn hơn 35 °C và độ cao không đổi (Led (on), còi (on)).



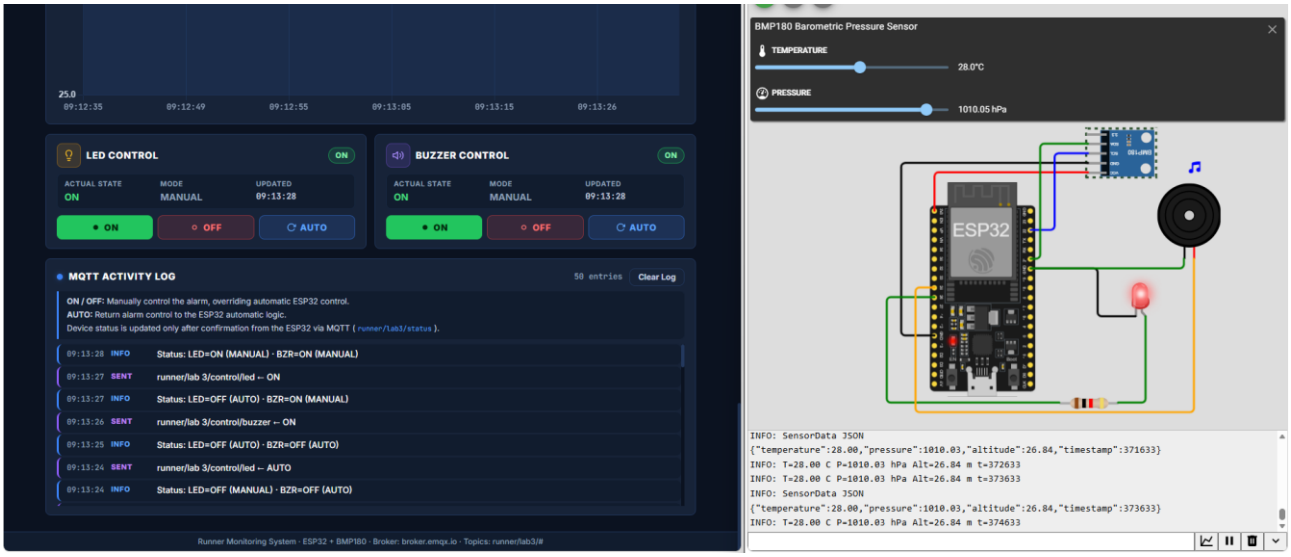
Hình 10. Nhiệt độ nhỏ hơn 10 °C và độ cao không đổi (Led (on), còi (on)).



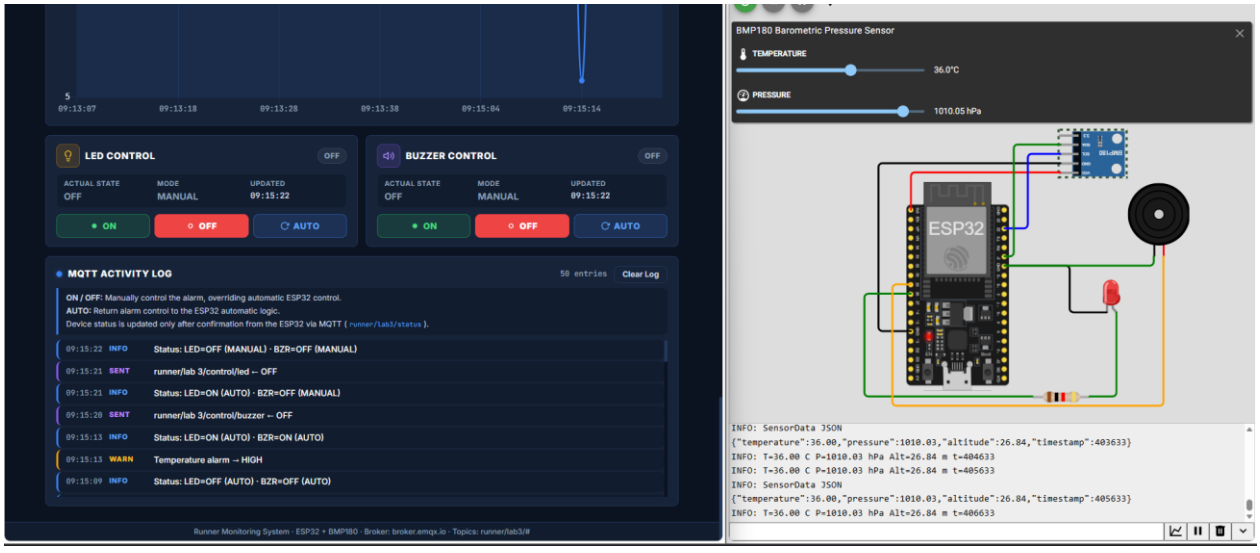
Hình 11. Nhiệt độ từ 10 °C đến 32 °C và Độ cao đổi từ 5 m đến dưới 10 m trong khoảng 5 giây (Led (on), còi (off)).



Hình 12. Nhiệt độ từ 10 °C đến 32 °C và Độ cao đổi từ 10 m trở lên trong khoảng 5 giây (Led (on), còi (on)).



Hình 13. Nhiệt độ từ 10 °C đến 32 °C và độ cao không đổi dùng chế độ manual bật (Led (on), còi (on)).



Hình 14. Nhiệt độ trên 32 °C và độ cao không đổi dùng chế độ manual tắt (Led (off), còi (off)).